

Corinne FAY

01/10/2026

Plan de validation

Produit : SOC Tracking

Table des matières

1. Introduction	4
2. Convertisseur Axi4s VGA	5
2.1. Architecture à valider	5
2.2. Validation du bloc - simulation.....	6
2.2.1. Test bench.....	6
2.2.2. Simulation.....	6
3. Pattern generator	7
3.1. Architecture à valider	7
3.1. Validation du bloc pattern generator - simulation	7
3.1.1. Test bench.....	7
3.1.2. Simulation.....	7
3.2. Validation du pattern generator – Intégration dans le top.....	8
3.2.1. Intégration dans le top design	8
3.2.2. Simulation vhdl.....	8
3.2.3. Test sur carte	8
3.2.4. Démo :.....	8
4. Filtrage.....	9
4.1. Architecture à valider	9
4.2. Validation d'un filtre : simulation	9
4.2.1. Test bench.....	9
4.2.1. Simulation.....	9
4.3. Validation du filtre gris.....	11
4.3.1. Architecture du filtre gris	11
4.3.1. Validation en simulation vhdl.....	11
4.3.2. Validation sur carte	11
4.4. Validation du calcul du filtre gauss	12
4.4.1. Architecture de calcul à valider	12
.....	13
4.4.2. Validation en simulation vhdl	13
4.4.3. Validation sur carte	13
4.5. Validation du calcul du filtre harris	13
4.5.1. Architecture de calcul à valider	13
4.5.2. Validation en simulation vhdl.....	15
4.5.3. Validation sur carte	15
4.6. Validation d'un filtre – Intégration dans le top	15
4.6.1. Intégration du filtre dans le top design entre le générateur de pattern et le convertisseur axi4s/VGA	15
4.6.2. Simulation vhdl.....	15
4.6.3. Test sur carte	16

4.6.4.	Démo :	16
5.	Ligne complète.....	16
5.1.	Architecture à valider	16
5.2.	Validation de la ligne – simulation	16
5.2.1.	Test bench.....	16
5.2.1.	Simulation vhdl.....	16
5.2.1.	Test sur carte	16
5.2.2.	Démo :	17
6.	DMA	17
6.1.	Architecture à valider	17
6.2.	Validation sur carte	17
6.2.1.	Environnement	17
6.2.2.	Programme de contrôle	17
6.2.3.	Mesures	18
7.	Architecture complète	19
7.1.	Architecture à valider	19
7.2.	Validation sous vitis	19
7.2.1.	Développement du code C	19
7.2.2.	Validation en mode demo des différents mode de fonctionnement	19

1. Introduction

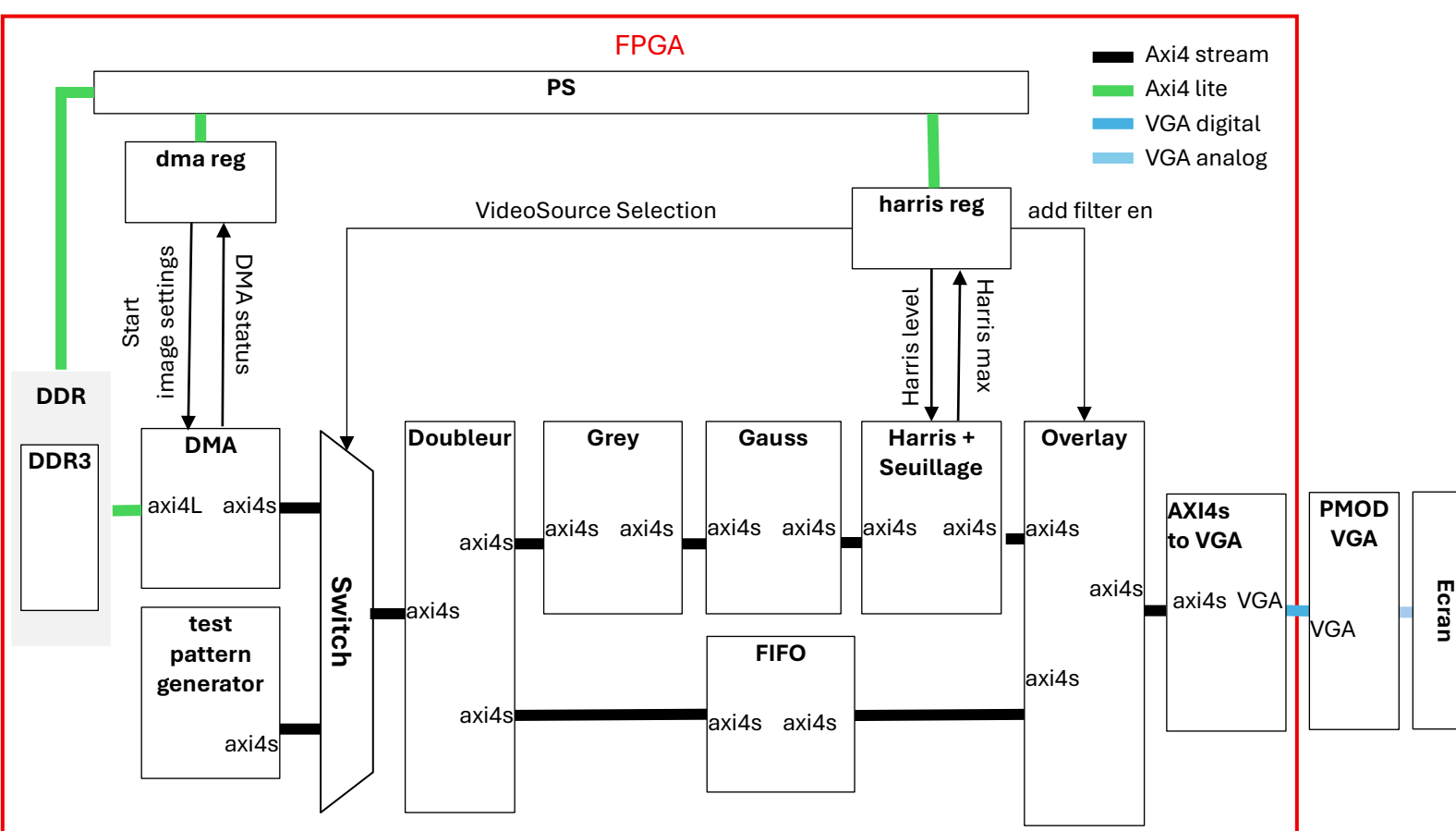
Ce plan de validation va détailler les étapes de validation de l'architecture ci-dessous

L'objectif est d'afficher sur écran VGA une image fournie par un fichier stocké en mémoire DDR ou fournie par un pattern generator.

Cette image pourra être affichée directement sans filtrage ou avec superposition d'un filtrage

Le filtrage est composé de plusieurs étages : filtre gris, filtre de gauss, filtre harris et seuillage . Le but est d'identifier les contours de l'image

Ce plan de validation permet de valider les blocs unitairement puis de valider leur intégration dans l'architecture complete.

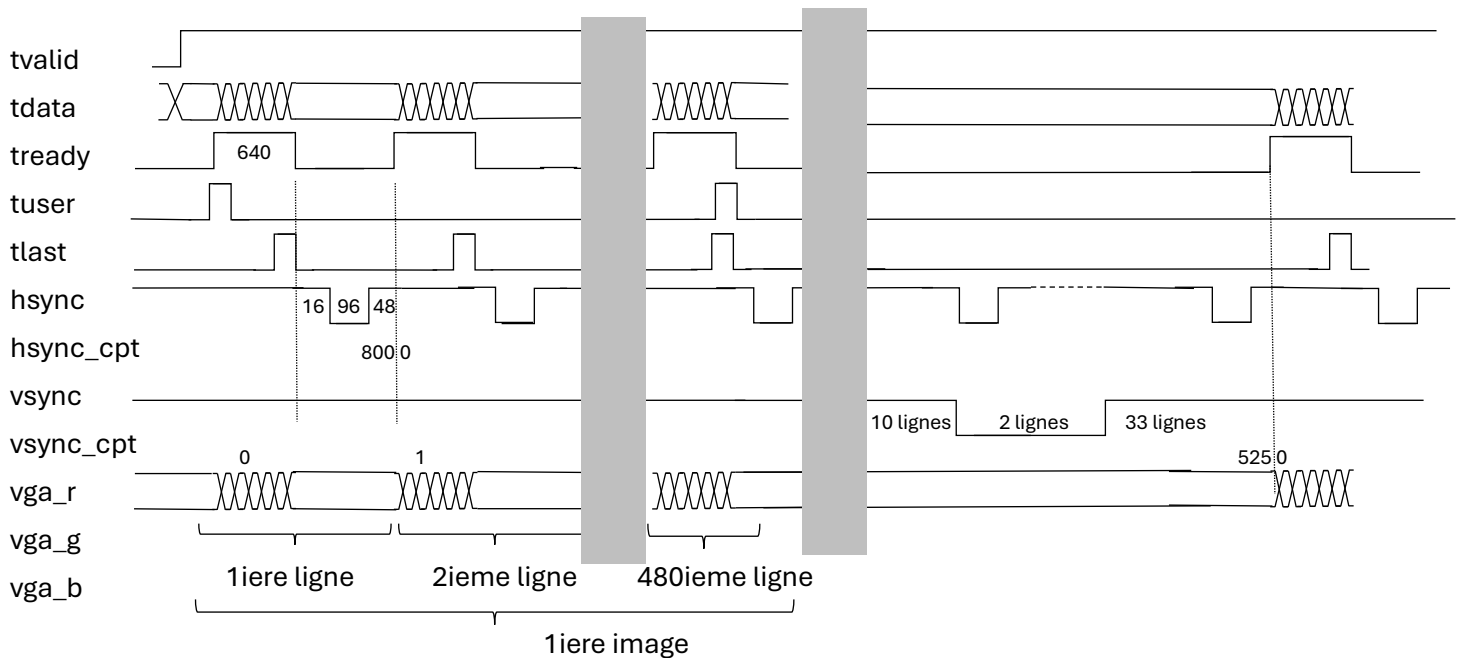
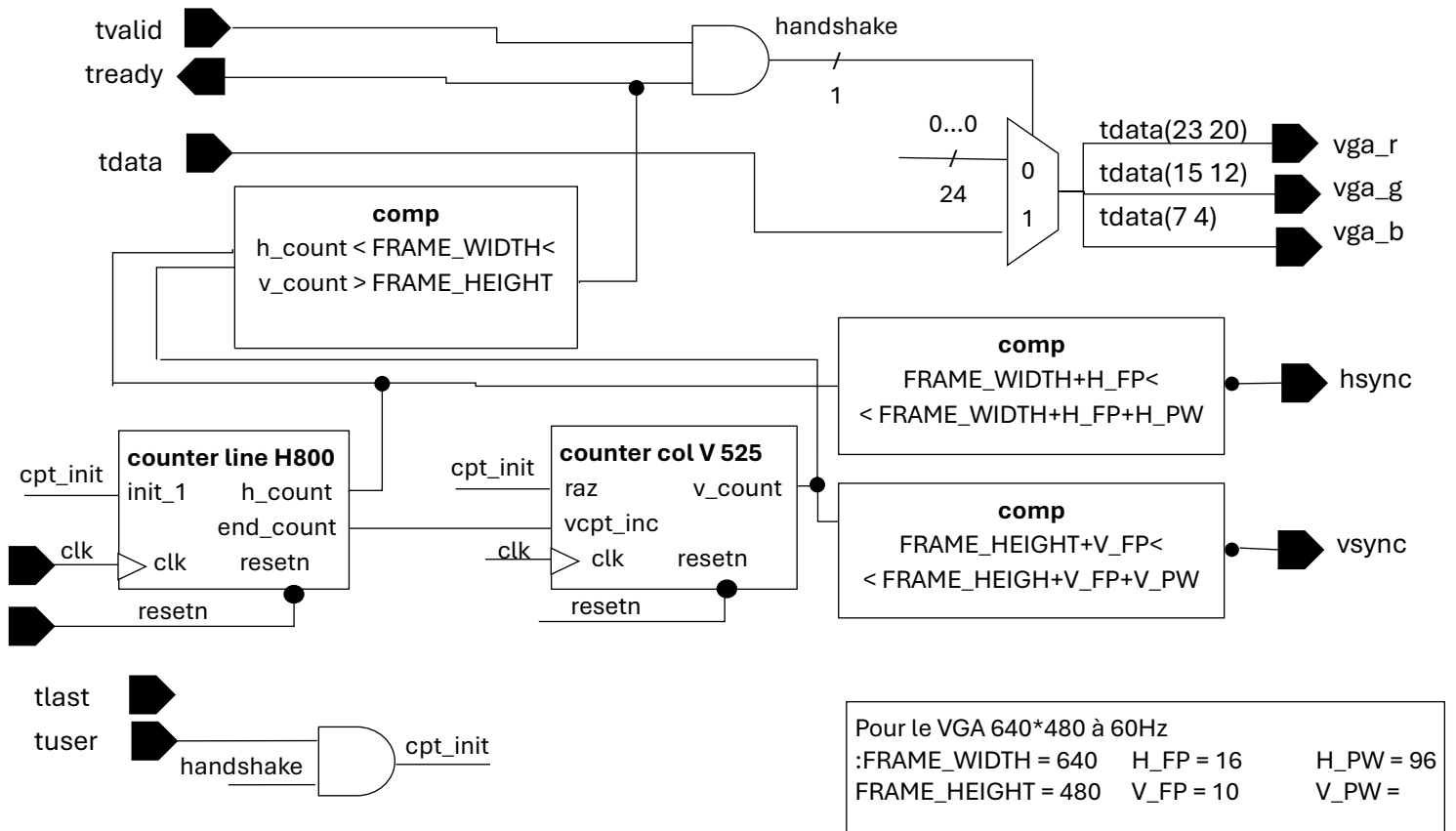


La validation va se dérouler dans l'ordre ci-dessous afin de pouvoir valider les blocs indépendamment puis intégrés dans le système.

- Convertisseur axi4s ti VGA
- Pattern generator
- Filtre gris
- Filtre de Gauss
- Filtre Harris
- Doubleurs, fifo et overlay
- DMA
- Système complet

2. Convertisseur Axi4s VGA

2.1. Architecture à valider



2.2. Validation du bloc - simulation

2.2.1. Test bench

Le test bench utilisé consiste à utiliser le générateur de trame axis4s fournit (axi4sdriver. vhd)
axi4sdriver fournit une trame axi4s à partir d'un fichier texte en entrée.

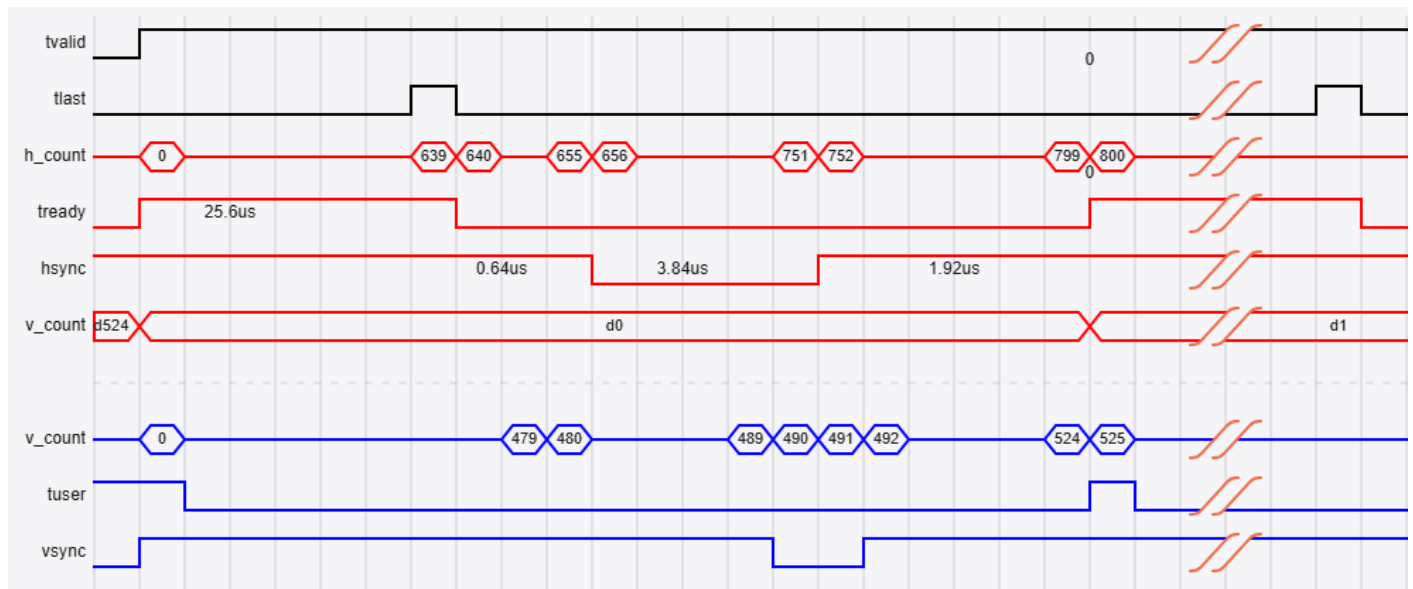
Le fichier texte doit contenir un table de 640*480 mots de 8bit en décimal

Afin de faciliter la validation du transfert des données, le fichier texte utilisé en entrée a été rempli de mots de 8 bits générés de façon aléatoire et de sorte que 2 nombres consécutifs ne soient pas identiques

Les test bench comptera le nombre de pixels et de lignes transmises en sortie du bloc

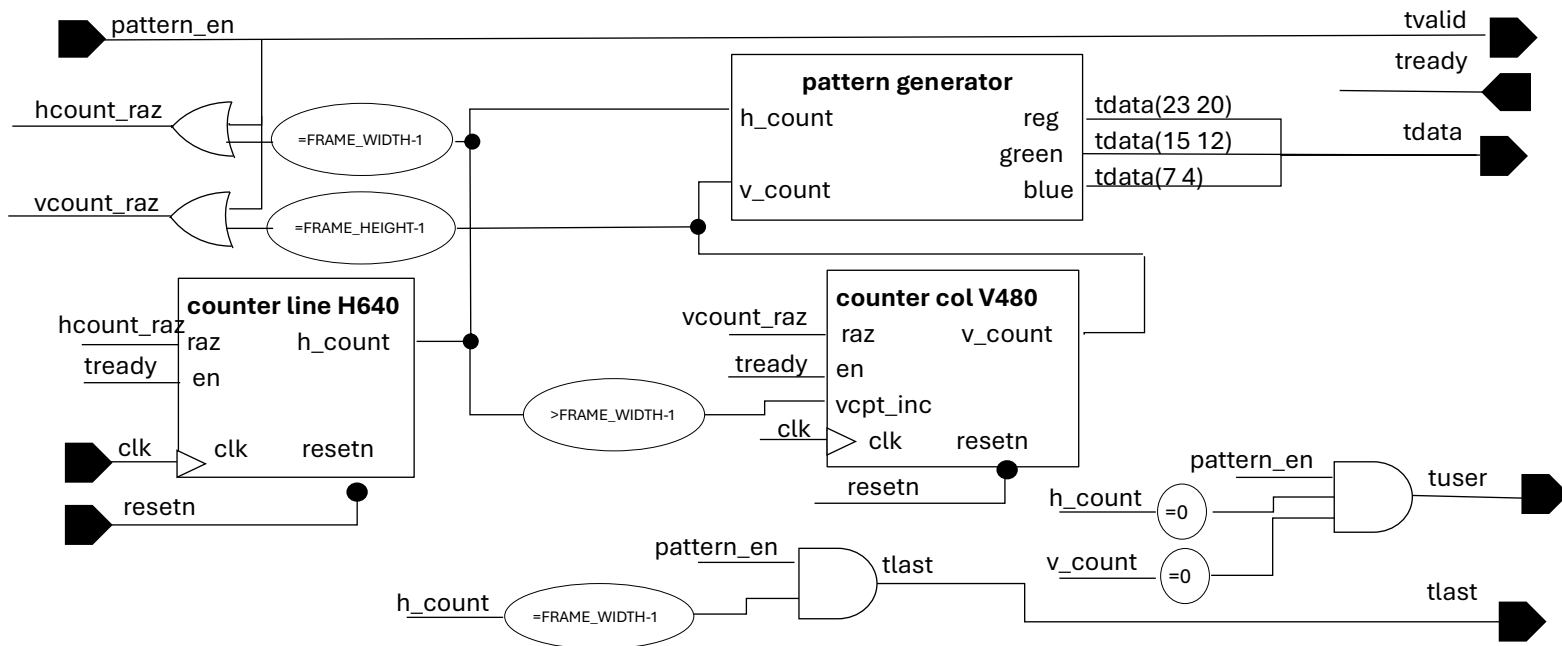
2.2.2. Simulation

- on vérifie les signaux VGA et les timings selon le chronogramme ci-dessus
- tuser= 1 sur le premier bit de data pendant 1 periode(clk) toutes les $((L+160) * H + 45) * \text{periode}(\text{clk})$
- tlast = 1 sur le dernier bit de data de la ligne pendant 1 periode(clk) toutes les $(L+160)*\text{periode}(\text{clk})$
- tdata sur 24 bits déroule les couleurs 8 bits pour chaque pixel, le transfert de tdata dure $L*\text{periode}(\text{clk})$
- tvalid = 1 pendant le transfert pour valider la transmission
- tready = 1 pendant le transfert des données c'est-à-dire $L*\text{periode}(\text{clk})$
- hsync dure $96*\text{periode}(\text{clk})$ passe à 0 $16*\text{periode}(\text{clk})$ après la fin de transmission des données
- vsync = 0 pendant $2*\text{periode}(\text{clk})$ toutes les fins d'image
- les signaux vga_r, vga_g, vga_b prennent pour valeur les msb de leur couleur respectives sur le bus tdata
- les données lues et transférées correspond et bien aux données des fichiers texte utilisés en entrée



3. Pattern generator

3.1. Architecture à valider



Le bloc de création des signaux red, green, blue a été repris du la demo pmod vga .

3.1. Validation du bloc pattern generator - simulation

3.1.1. Test bench

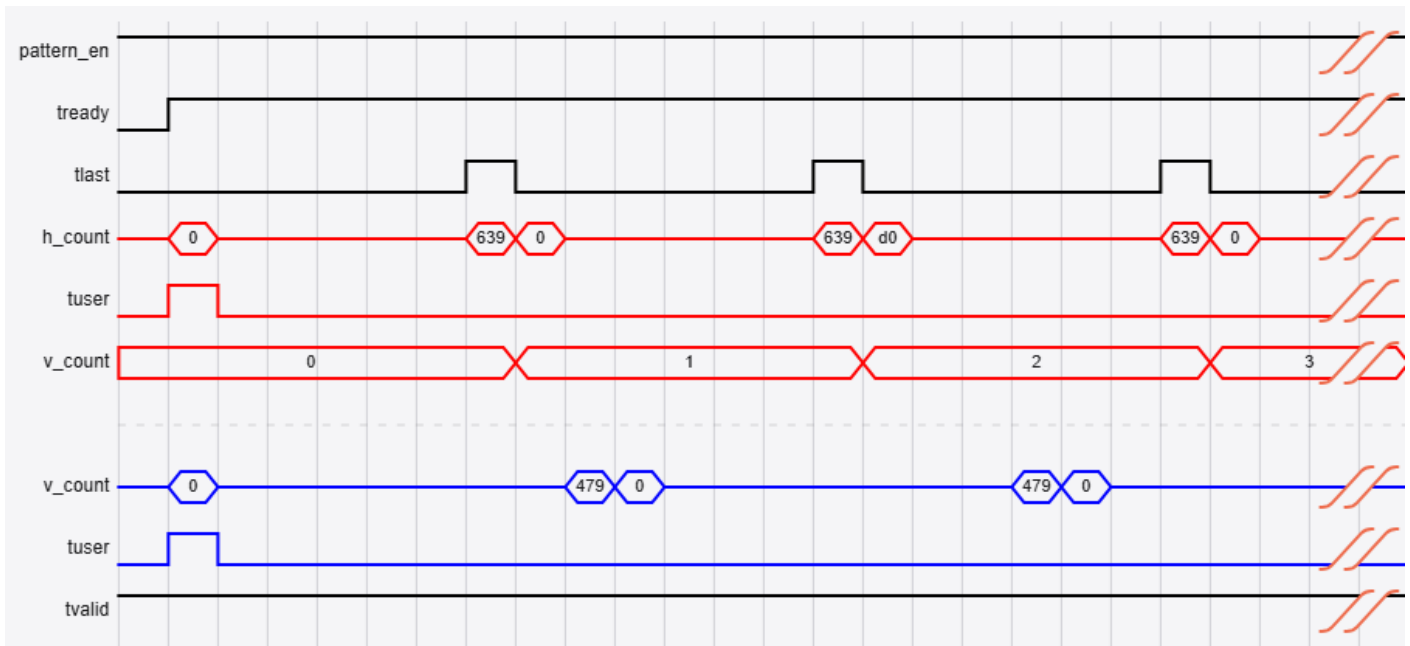
Le test bench utilisé consiste démarrer le pattern sur `pattern_en` et autoriser ou non la transaction sur `tready`

La sortie axistream du patter generator est connectée à 3 modules axis monitor fournis. Ces modules permettent de simuler un reception axi stream. Ils enregistrent la trame sur 8 bit et la convertissent en fichier .txt

Le pattern généré à partir de la demo du pmod vga contient des zones noires et des zones blanches qui compliquent la validation du transfert des data. Afin d'avoir un flux de data non nul sur l'axi stream , un pattern (type valeur de compteur) est créé.

Le pattern type pmod vga ne sera remis qu'a la fin de la validation du bloc pour l'affichage sur écran car il contient une partie fixe et une partie mobile.

3.1.2. Simulation

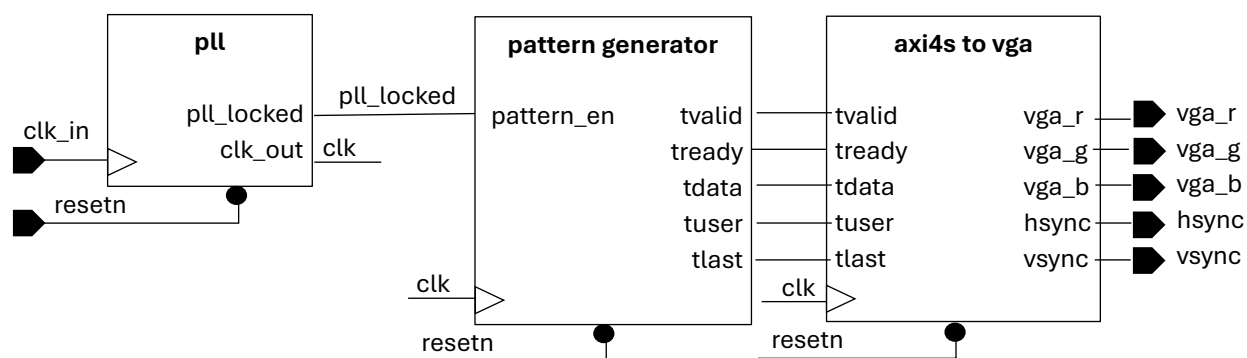


- on vérifie les signaux axistream et les timings selon le chronogramme ci-dessus
- tuser= 1 sur le premier bit de data pendant 1 periode(clk) toutes les $(L * H) * \text{periode}(\text{clk})$
- tlast = 1 sur le dernier bit de data de la ligne pendant 1 periode(clk) toutes les $L * \text{periode}(\text{clk})$
- tdata sur 24 bits déroule les couleurs 8 bits pour chaque pixel, le transfert de tdata dure $L * \text{periode}(\text{clk})$
- tvalid = 1 tant que le pattern est enable
- tready = 1 pendant le transfert des données est autorisé
tready = 0 : les compteurs sont arrêtés;
tready = 1 le transfert reprends sans perte de données

3.2. Validation du pattern generator – Intégration dans le top

3.2.1. Intégration dans le top design

Ce top design comprend le générateur de pattern et du convertisseur axi4s/VGA et une pll qui génère la clock de 25Mhz à partir de la 125Mhz présente sur la carte



3.2.2. Simulation vhd

- Simulation de la ligne ainsi formée
- vérification du format VGA avec une com axi4s générée par le pattern generator

Vérification des timings vga (cf. 2.2.2)

3.2.3. Test sur carte

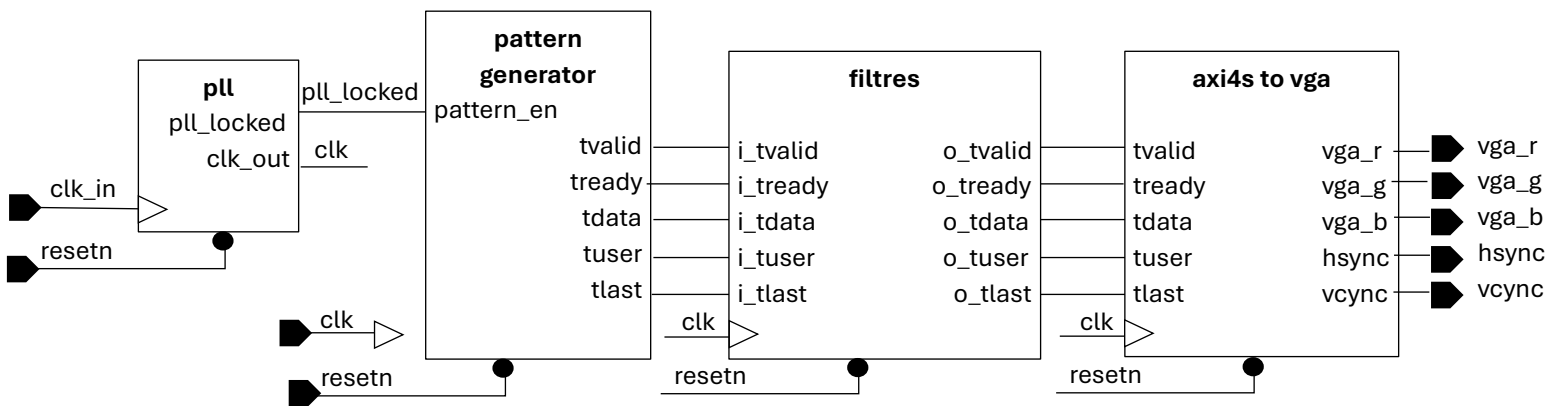
Après synthèse et implémentation, le circuit est testé sur carte.

- mesure sur ila du signal axi4s de sortie
- mesure des sortie VGA sur ila
- mesure du signal hsync/vsync à l'oscillo

3.2.4. Démo :

visualisation sur l'écran du pattern généré. vérification visuelle du bon alignement de l'image

4. Filtrage



4.1. Architecture à valider

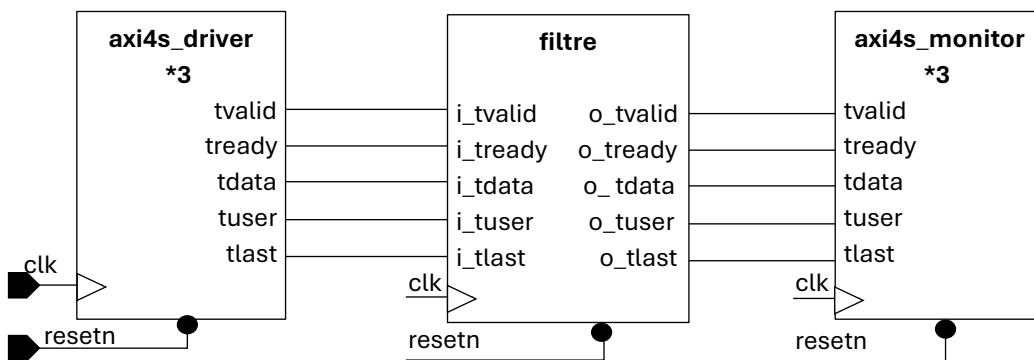
La chaine de filtrage va comporter 3 filtres:

gris
gauss
harris

Chaque filtre sera validé seul puis il sera ensuite intégré à la chaine complète pour une validation top.

4.2. Validation d'un filtre : simulation

4.2.1. Test bench



Le test bench est composé de

3 axis4 driver qui lisent chacun un fichier d'entrée et sortent une trame axi stream correspondant à ce fichier d'entrée

3 axis4 monitor qui créent chacun un fichier de sortie correspondant à la trame axi stream reçue.

Au centre le filtre fait son calcul sur les datas avant de les renvoyer

Les fichiers d'entrée de l'axi4 driver contiendront une table de 640*480 mots de 8 bit en décimal de valeurs adjacentes différentes (pour faciliter la vérification et la validation de la trame)

Chaque axi4s_driver et axi4s_monitor traite une couleur de l'image au format 8bits .

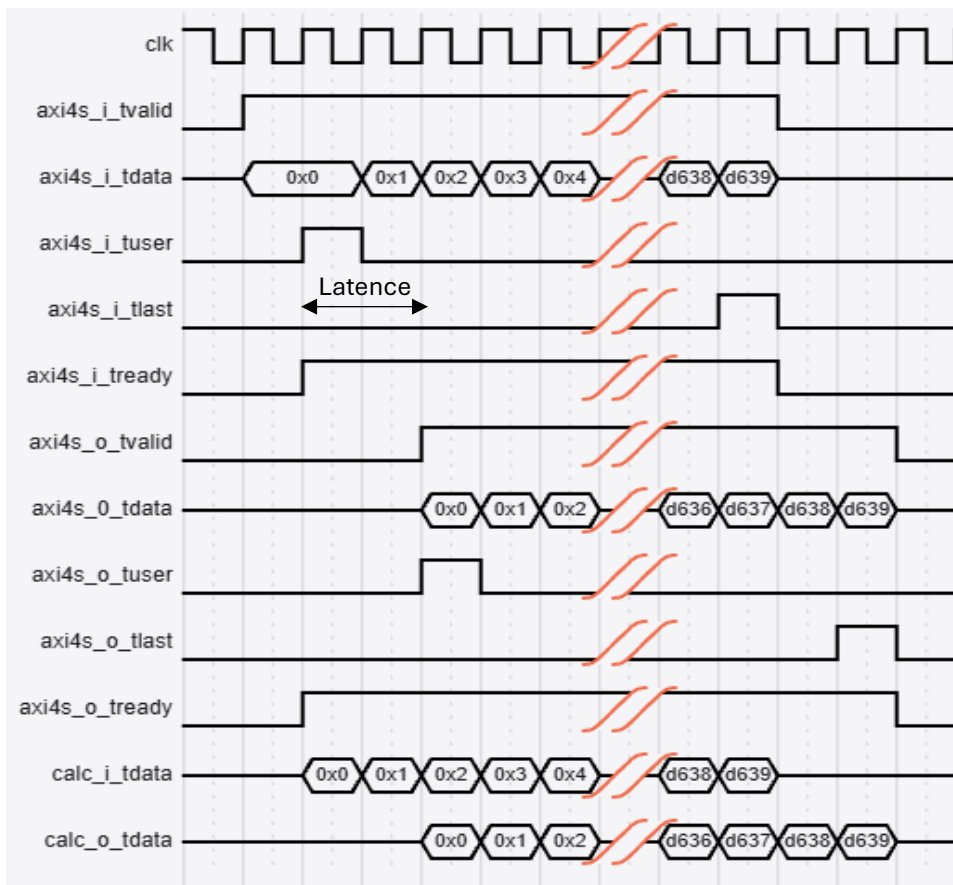
Les fichiers lus par les axis4_drivers texte contiendront un table de 640*480 mots de 8bit en décimal

4.2.1. Simulation

- Validation du calcul du filtre

Les fichiers de sortie des axi4s_monitor sont comparés à des fichiers théoriques de résultat de filtrage calculés. Si les fichiers sont identiques : la calcul est juste

- Validation du protocole axi stream



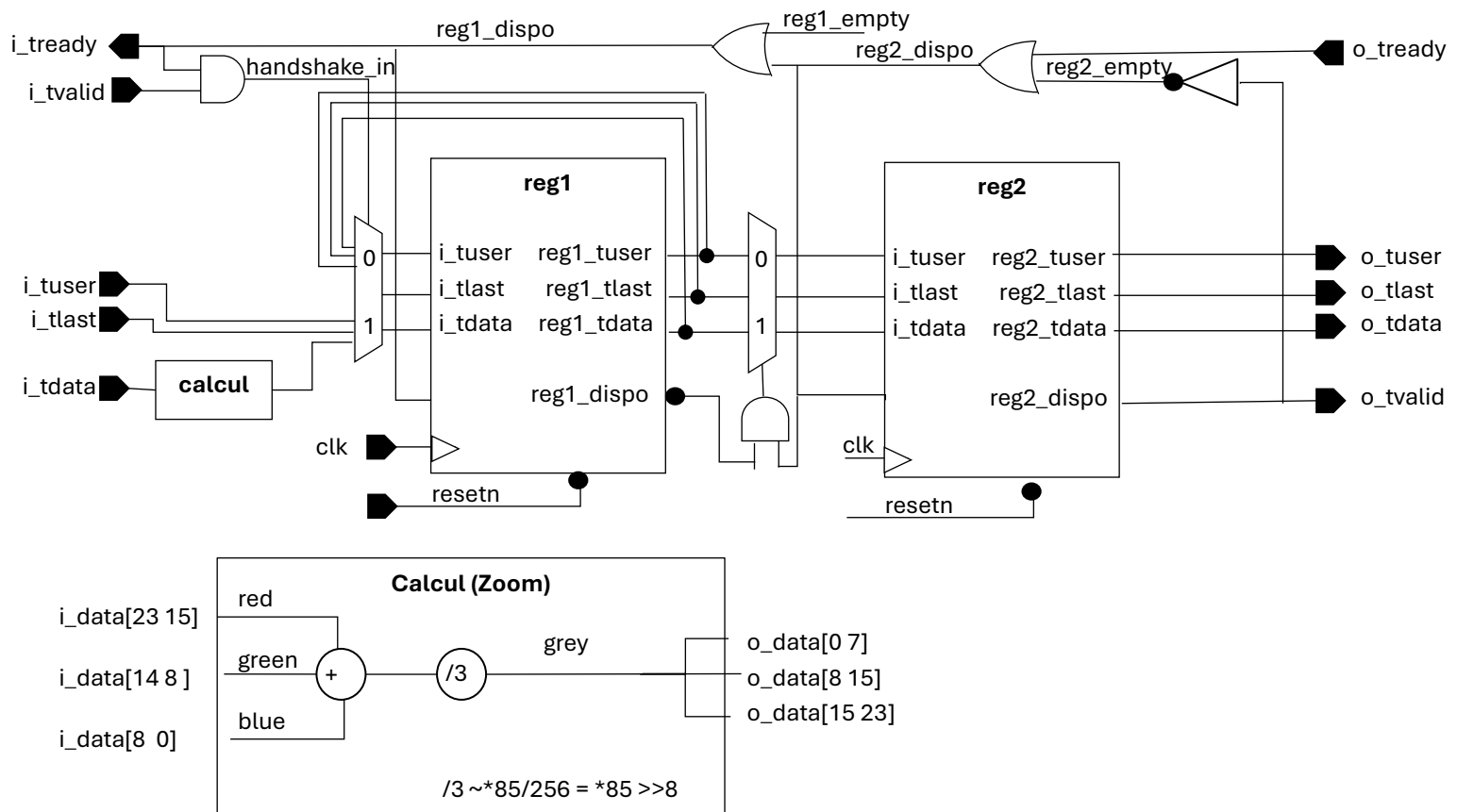
La validation se fera suivant ce chronogramme.

On tiendra compte de la latence dans le calcul du filtrage et dans l'envoi des données

- La première donnée reçue est bien la première donnée envoyée sur le tuser.
- Chaque fin de ligne est bien identifiée par le tlast, la dernière donnée de la ligne est bien envoyée à ce moment là
- Vérifier l'état des compteurs à chaque transition de fin et début de ligne et de frame

4.3. Validation du filtre gris

4.3.1. Architecture du filtre gris



4.3.1. Validation en simulation vhd1

Le table de sortie du test bench est comparée à la table calculée

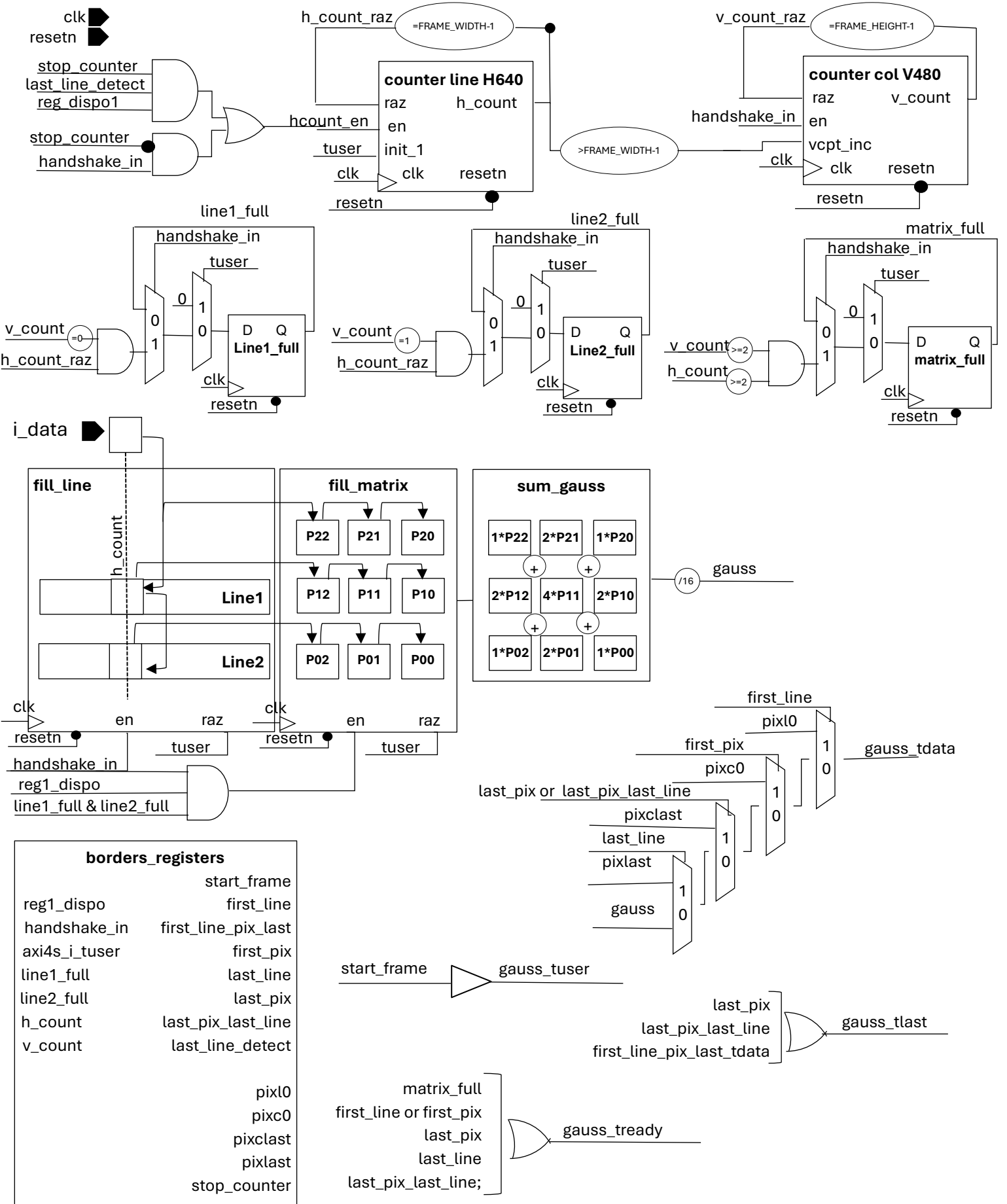
Le séquencement de remplissage des registres est vérifié

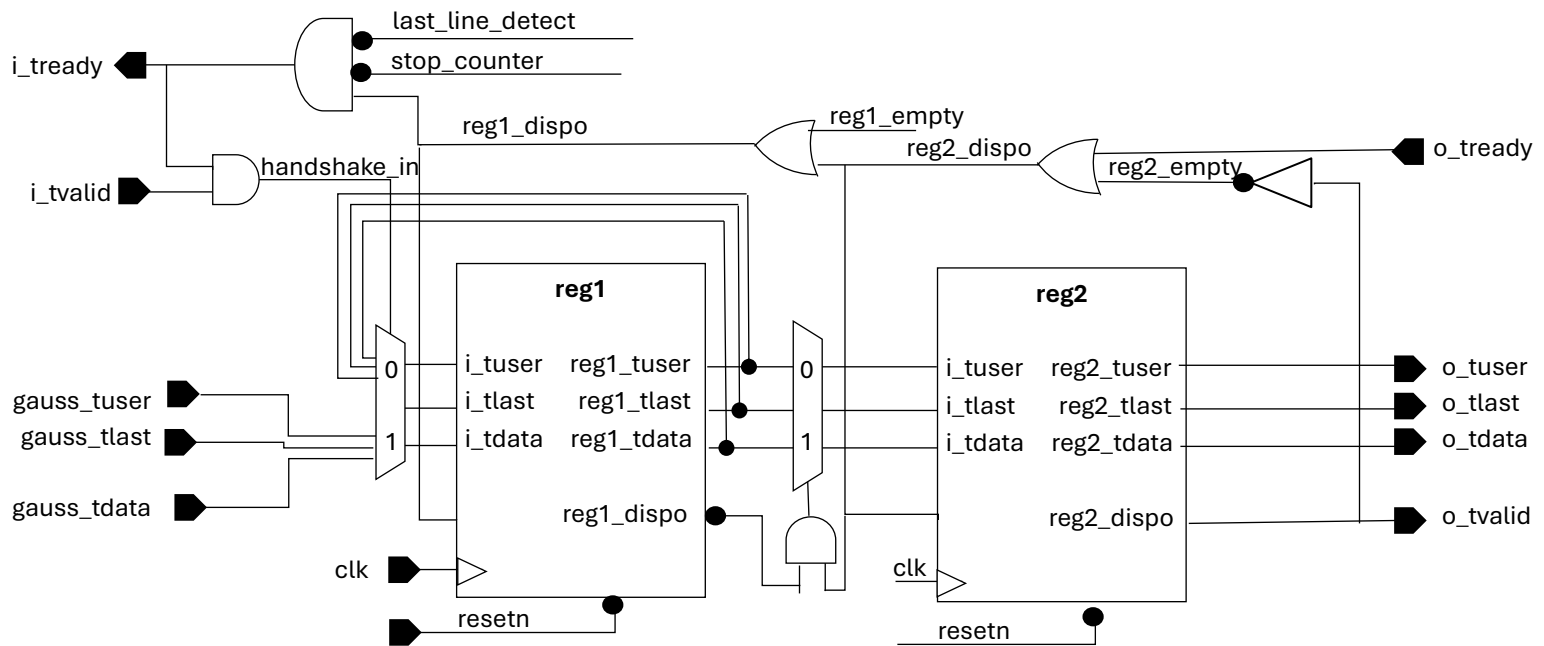
4.3.2. Validation sur carte

Le pattern est affiché en niveaux de gris

4.4. Validation du calcul du filtre gauss

4.4.1. Architecture de calcul à valider





4.4.2. Validation en simulation vhdl

La table de sortie du test bench est comparée à la table calculée

Le séquençement du calcul du filtrage est vérifiée sur les débuts et fins de frame et sur les débuts et fins de ligne.

Vérifier le remplissage des line buffers et de la matrice de calcul

Vérifier que tous les pixels d'entrée sont transmis après le filtrage.

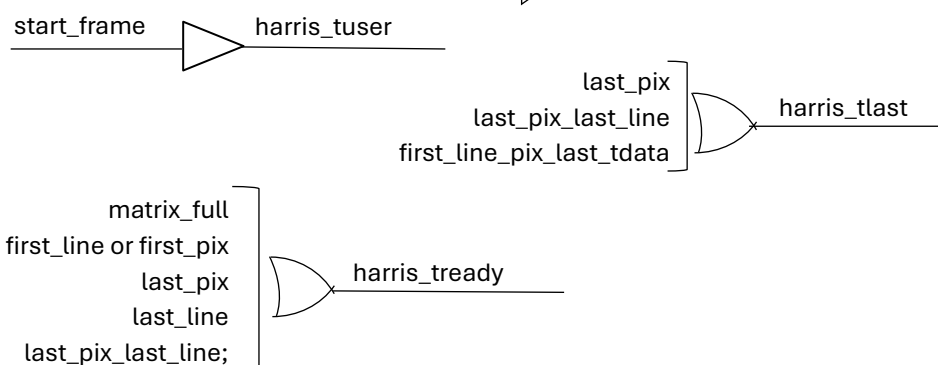
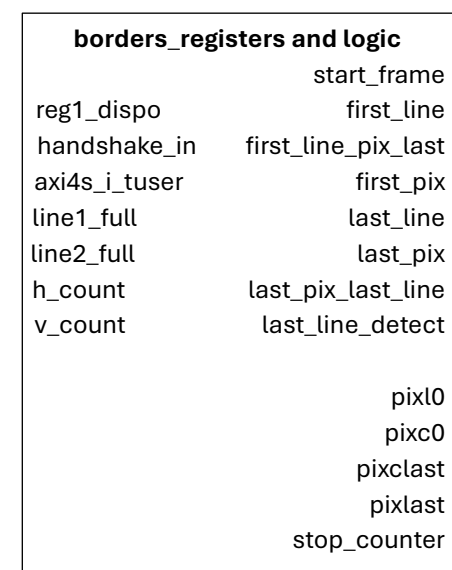
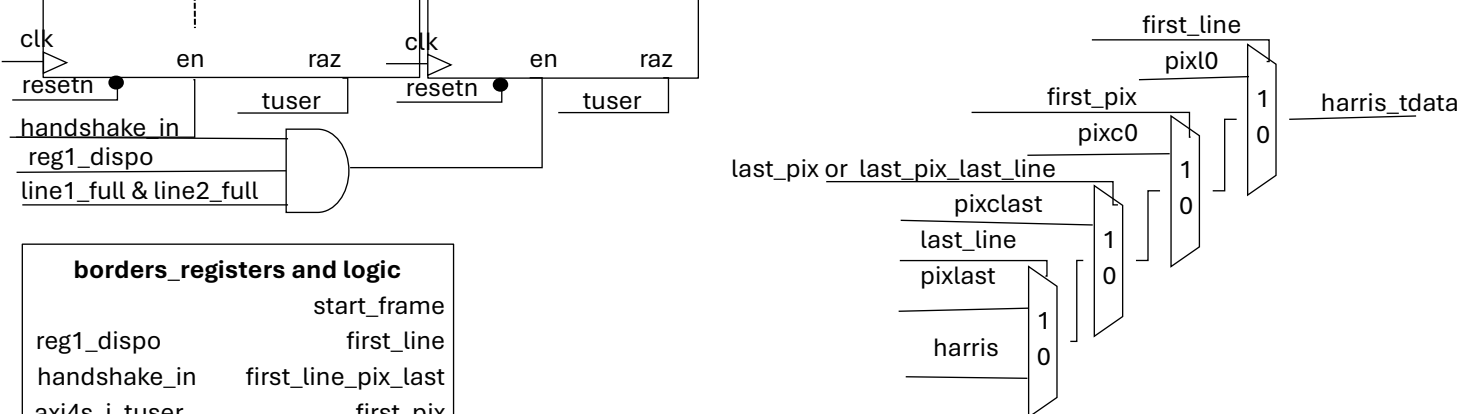
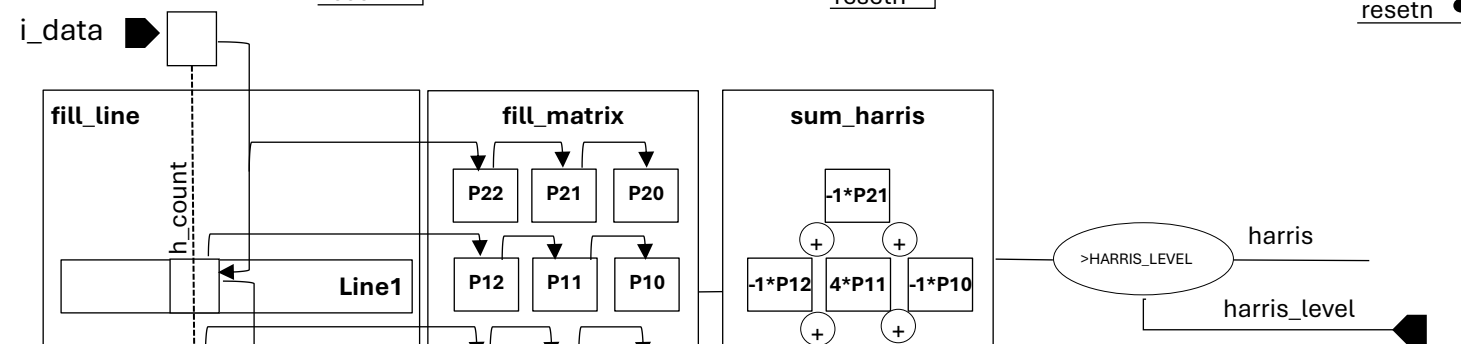
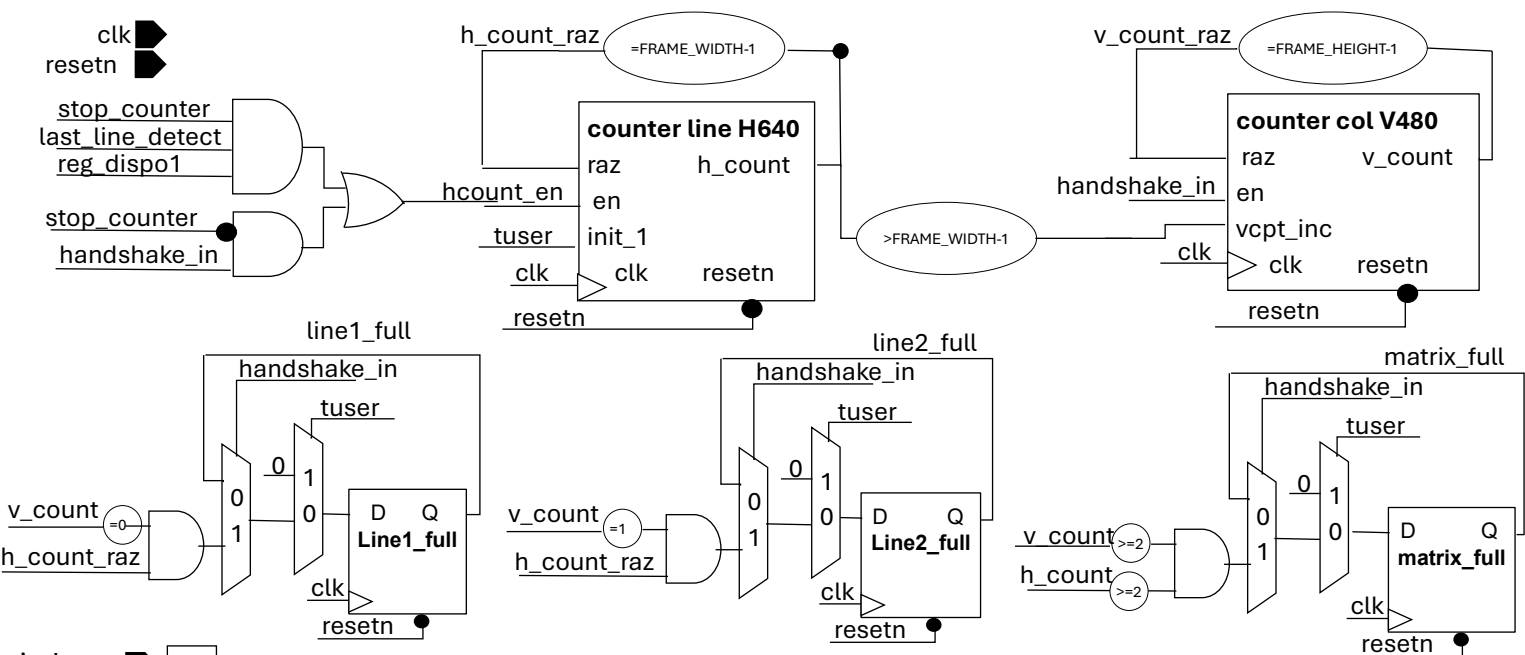
4.4.3. Validation sur carte

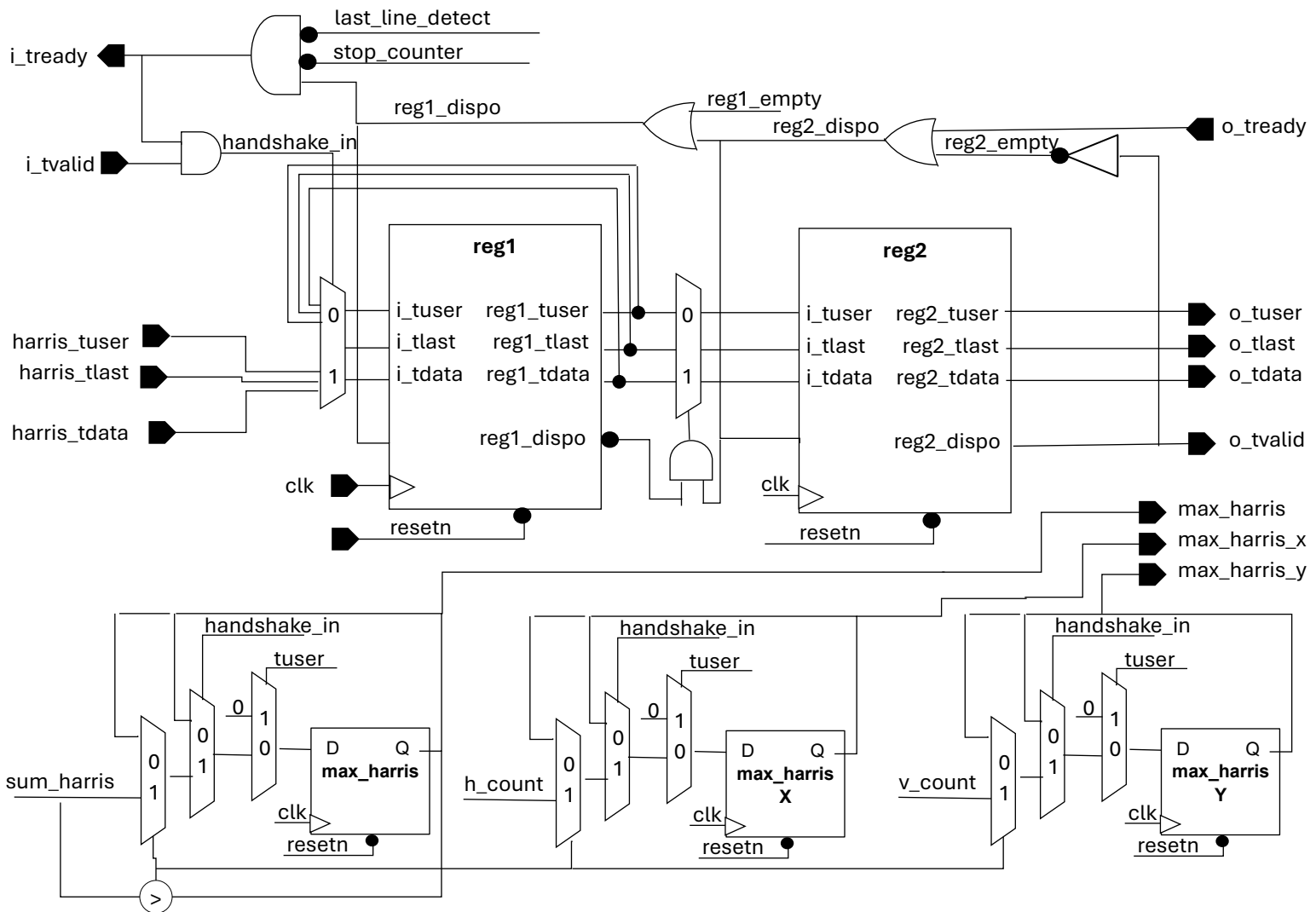
Le pattern est affiché en niveaux de gris flou

Vérifier visuellement s'il n'y a pas de décalage dans l'affichage du pattern

4.5. Validation du calcul du filtre harris

4.5.1. Architecture de calcul à valider





4.5.2. Validation en simulation vhdl

La table de sortie du test bench est comparée à la table calculée

Le séquençement du calcul du filtrage est vérifiée sur les débuts et fins de frame et sur les débuts et fins de ligne.

Vérifier le remplissage des line buffers et de la matrice de calcul

Vérifier que tous les pixels d'entrée sont transmis après le filtrage.

4.5.3. Validation sur carte

Le pattern est affiché en niveaux de gris flou

Vérifier visuellement s'il n'y a pas de décalage dans l'affichage du pattern

4.6. Validation d'un filtre – Intégration dans le top

4.6.1. Intégration du filtre dans le top design entre le générateur de pattern et le convertisseur axi4s/VGA

4.6.2. Simulation vhdl

- Simulation de la ligne ainsi formée
- Vérification du format VGA
- Vérification des timings vga (cf 2.2.2)
- Vérification du protocole axi stream sur toute la chaine

- Vérification du transfert du premier pixel de début de frame
- Vérification du transfert du dernier pixel de fin de ligne
- Vérification du transfert du premier et dernier pixels au changement de frame

4.6.3. Test sur carte

Après synthèse et implémentation, le circuit est testé sur carte.

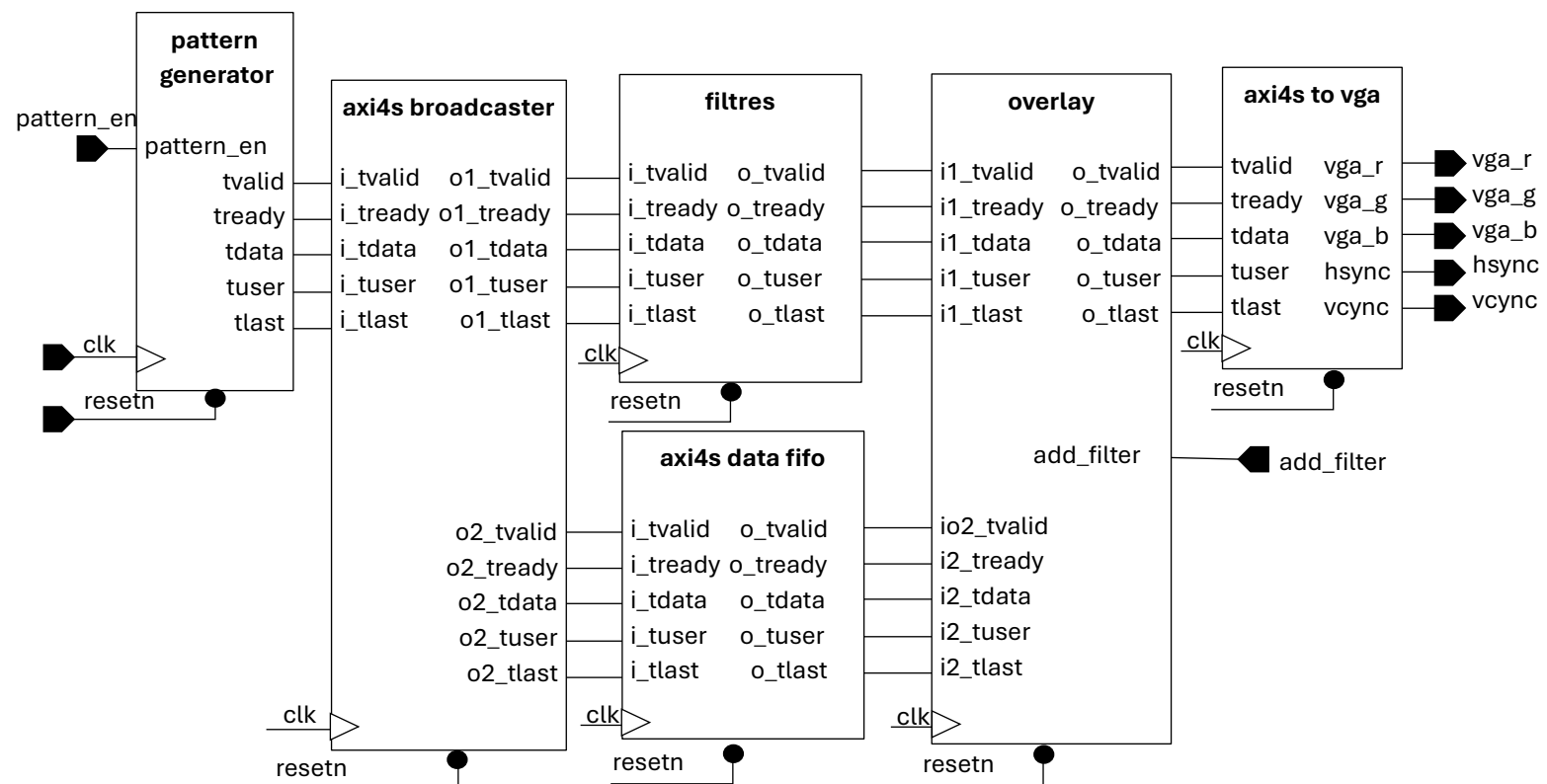
- Mesure sur ila du signal axi4s de sortie
- Mesure des sortie VGA sur ila
- Mesure du signal hsync/vsync à l'oscillo

4.6.4. Démo :

- Visualisation sur l'écran de l'image filtrée.
- Vérification visuelle du bon alignement de l'image

5. Ligne complète

5.1. Architecture à valider



5.2. Validation de la ligne – simulation

5.2.1. Test bench

La simulation est lancée sur le `pattern_en` pendant 3 cycles de frame, on ajoute le filtre (`add_filter`) pendant la deuxième frame

5.2.1. Simulation vhd

- Simulation de la ligne ainsi formée
- Vérification du format VGA
- Vérification des timings vga (cf 2.2.2)
- Vérification du protocole axi stream sur toute la chaine

5.2.1. Test sur carte

Le signal `add_filter` est routé sur un bouton pour pouvoir le tester facilement sur carte

Après synthèse et implémentation, le circuit est testé sur carte.

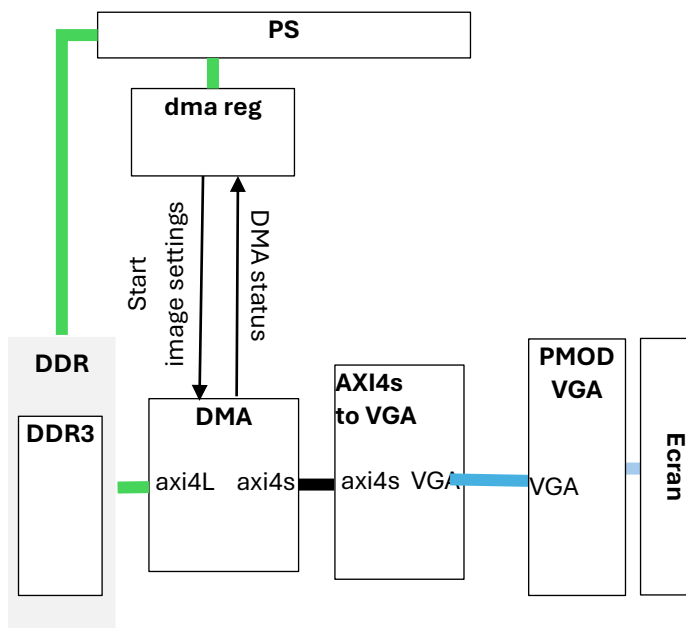
- Mesure sur ila du signal axi4s de sortie
- Mesure des sortie VGA sur ila
- Mesure du signal hsync/vsync à l'oscillo

5.2.2. Démo :

- Visualisation sur l'écran de l'image initiale et de l'image avec l'overlay du filtre.
- Vérification visuelle du bon alignement de l'image

6. DMA

6.1. Architecture à valider



Afin de valider le fonctionnement du DMA , nous avons besoin du processing system , de la DDR et de registres pour les signaux de contrôle du DMA.

La validation se fera directement sur carte sous vitis.

Le circuit sera mis en place à partir du référence design fournit et le DMA fournit

6.2. Validation sur carte

6.2.1. Environnement

Le démarrage du circuit se fait sous vitis, et en parallèle vivado peut être activé pour faire les mesures sur l'ila.

6.2.2. Programme de contrôle

Le programme de contrôle en C permet d'activer le DMA en écrivant dans ses registres de contrôle .

Table de registres du DMA sur 32 bits

Nom du registre	Descriptif		ADRR OFFSET
DMA_REG_IMAGE_START	B0 : Start DMA	W	0x00 SLV_REG0
DMA_REG_IMAGE_W	B11 to B0 : Largeur de l'image	W	0x04 SLV_REG1
DMA_REG_IMAGE_H	B11 to B0: Hauteur de l'image	W	0x08 SLV_REG2
DMA_REG_IMAGE_LO	LO adresse de départ de l'image dans la DDR	W	0x0C SLV_REG3
DMA_REG_IMAGE_HI	HI adresse de départ de l'image dans la DDR	W	0x10 SLV_REG4
DMA_REG_STATUS	B2 : DMA ready B1: DMA idle B0: DMA done	R	0x14 SLV_REG5

Le chargement de l'image en DDR se fait par commande dans la console xsct.
la relecture de cet emplacement mémoire permet de vérifier la bonne écriture de l'image

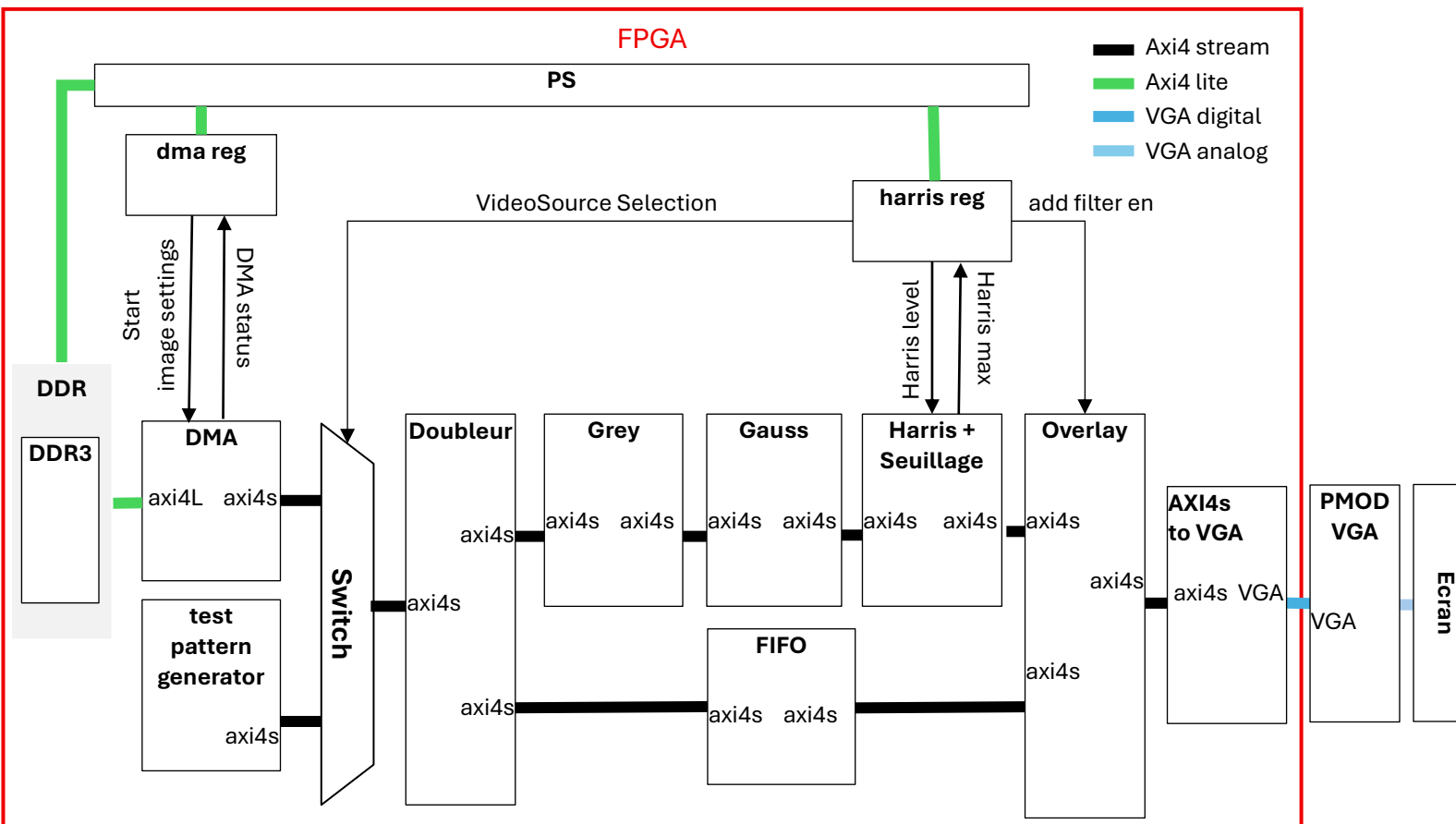
6.2.3. Mesures

Au chargement de l'image en DDR si le DMA est activé, l'image apparait à l'écran .
La mesure des signaux vga et axi stream peut se faire sur l'ILA.

7. Architecture complète

Tous les blocs ont été validé séparément , nous pouvons passer à la mise en place de la validation de l'architecture complète

7.1. Architecture à valider



Les chemins à valider sur ce top_level sont

Affichage de l'image fournie par le DMA ou par le pattern

Affichage d'une image sans traitement ou avec superposition du traitement

L'image doit pouvoir être stockée dans la DDR par lecture de carte SD

7.2. Validation sous vitis

7.2.1. Développement du code C

Le code C doit pouvoir communiquer avec l'UART afin de proposer un menu de test et de vérification de la plateforme

Menu

1. Lecture et mise en mémoire DDR de l'image stockée sur la carte SD
2. Démarrage du DMA , source d'image par défaut
3. Choix de la source d'image
 - a. DMA
 - b. Pattern generator
4. Mode d'affichage
 - a. Image sans traitement
 - b. Image avec overlay harris
5. Modification du niveau de filtrage harris
6. Lecture du statut du DMA
7. Lecture de la réponse maximale en sortie du filtre harris

7.2.2. Validation en mode demo des différents mode de fonctionnement

Table de registres du filtre harris sur 32 bits

Nom du registre	Descriptif		ADRR OFFSET
HARRIS_REG_MAX	B10 to B00 : réponse maximale du filtre	R	0x00 SLV_REG0
HARRIS_REG_MAX_X	B9 to B0 : coordonnée h du max origine en haut a gauche de l'écran	R	0x04 SLV_REG1
HARRIS_REG_MAX_Y	B9 to B0: coordonnée v du max origine en haut a gauche de l'écran	R	0x08 SLV_REG2
HARRIS_REG_LEVEL	B9 to B0 seuil du filtre harris	W	0x0C SLV_REG3
HARRIS_REG_OVERLAY	B0: 1 : ajout de l'overlay sur l'image 0 : image sans filtrage B1 1 pattern en 0 pattern etteint	W	0x10 SLV_REG4